

Physicalized Model Weights - cycles 29-31

2026-05-13

Contents

Physicalized Model Weights - cycles 29-31	1
Abstract	1
Introduction	1
Approach	2
Findings	2
Discussion	6
Open Questions	6
References	7
Appendix: Implementation Details	7

Physicalized Model Weights - cycles 29-31

Abstract

Cycles 29-31 extended the closed physicalized-model-weights campaign with three post-closure checks: a local HDL toolchain condition probe, a cross-artifact invariant checker, and a public programmable-baseline recency screen. These cycles did not reopen the scientific conclusion. The current campaign endpoint remains unchanged: no current physicalized-weight performance or economic superiority claim, no actual measured reopen candidate, no new reopen gate, and no current artifact that reopens the Phase 2 downgrade.

Cycle 29 validated M-TOOLCHAIN-1, which refreshed prototype-verification evidence using available Verilator, Yosys, and Graphviz checks. Compiled Verilator simulation remained blocked by the local environment because make and a C++ compiler were unavailable. Cycle 30 validated M-INVARIANT-1, which checked endpoint-invariant consistency across canonical summaries and reports and found zero contradictions. Cycle 31 validated M-PUBLICBASE-1, which found that MLCommons MLPerf Inference v6.0, published on 2026-04-01, is newer than the campaign's earlier public MLPerf reference set and material enough to recommend a future programmable-baseline refresh, but not a physicalized-weight reopen [10]-[14].

Introduction

The campaign entered cycles 29-31 after a validated closure and archive package. Earlier cycles had already rejected full fixed frontier-model physicalization under current evidence, falsified safety/filter performance or economic superiority against stronger programmable baselines, and retained the hybrid prototype only as architecture, failure-mode, verification, and evidence-gating work.

The work in this cycle range therefore did not look for a new physicalized-weight win. It checked three bounded questions that remained legitimate after closure:

1. Whether local HDL tooling had changed enough to strengthen prototype verification.
2. Whether generated reports and summaries consistently represented the closed endpoint.

3. Whether newer public programmable-accelerator benchmark releases should trigger a future baseline refresh.

The validated future reopen condition remains the Phase 4 measured-evidence path: a lifecycle-valid measured hybrid production, shadow, or canary package with provenance, privacy, workload, baseline, threshold, and uncertainty requirements. Public benchmark data and local prototype checks do not satisfy that path.

Approach

The reporter source set for this report consisted of the nine provided cycle sessions, the generated artifacts, the plan and ledger, REFERENCES.md, and the updated workspace manifest.

Cycle	Milestone	Primary purpose	Source sessions
29	M-TOOLCHAIN-1	Check current HDL/toolchain capability and refresh prototype verification only	researcher 6e9bf19a, worker 8f6dfd72, auditor 03c7cbc6
30	M-INVARIANT-1	Check cross-artifact consistency of endpoint invariants	researcher e48b5978, worker a48d9de1, auditor 8347638a
31	M-PUBLICBASE-1	Screen official public programmable-baseline recency and materiality	researcher 6b47f02d, worker 52232565, auditor c445fb5f

A “compiled Verilator simulation” means translating the SystemVerilog hardware design into a C++ simulation binary and running it against golden vectors. A “golden vector” is a known input-output pair used to check that an implementation preserves expected behavior. An “endpoint invariant” is a campaign-wide field that must remain fixed unless real reopening evidence appears, such as `current_superiority_claim_count = 0`.

Findings

Cycle 29: Toolchain Condition Probe

Cycle 29 created M-TOOLCHAIN-1, a narrow probe of local HDL verification capability. The worker built `physicalized-weights/scripts/toolchain_condition_probe.py`, tests, a report, a capability matrix, a summary JSON, a figure, and refreshed tool logs.

The probe found that Verilator, Yosys, and Graphviz were available, but `make` and a C++ compiler were not. As a result, the compiled Verilator simulation remained `blocked_environment`, not failed. The available checks passed:

Check	Result
Verilator availability	available, Verilator 5.020
Yosys availability	available, Yosys 0.33
Graphviz dot availability	available, Graphviz 2.43.0

Check	Result
make	missing
C++ compiler	missing
Verilator lint	passed
Yosys eval	passed
Yosys synthesis	passed
Graphviz artifact check	checked
Compiled Verilator simulation	blocked by environment



Figure 1: local HDL/toolchain capability and verification coverage for the safety-filter prototype, distinguishing passed checks from environment-blocked compiled simulation.

The summary recorded `compiled_verilator_available: false`, `compiled_verilator_status: blocked_environment`, `compiled_verilator_missing_tools: ["make", "cxx_compiler"]`, and `compiled_verilator_equivalence_passed: null`. It also recorded `prototype_claim_reopened: false`, `performance_claim_reopened: false`, `current_superiority_claim_count: 0`, `actual_reopen_candidate_count: 0`, and `new_reopen_gate_count: 0`.

The auditor validated the milestone with no critical or moderate findings. Two minor points were noted: the generated PNG was a simple raster without embedded text labels, and the report emphasized Yosys eval and Graphviz while Yosys synthesis was present in the matrix, summary, and logs. No auditor patches were applied.

Cycle 30: Campaign Invariant Checker

Cycle 30 created M-INVARIANT-1, a deterministic consistency checker over the campaign's canonical summaries and reports. The goal was not to create a new scientific test. It was a quality-assurance layer to ensure that later handoff, archive, prototype, and closure artifacts did not contradict the validated endpoint.

The checker covered 17 artifacts: 8 JSON summaries and 9 Markdown reports. It checked fields such as `current_superiority_claim_count`, `actual_reopen_candidate_count`, `new_reopen_gate_count`, `current_artifacts_reopen`, and `performance_claim_reopened` wherever those fields were owned by an artifact.

The result was:

Metric	Value
Artifacts checked	17
JSON summaries checked	8
Markdown reports checked	9
Matrix rows	49
Consistent rows	23
Field absent or not applicable rows	19
Machine-readable contradictions	0
Warning-level ambiguous text rows	7
Introduced new gate	false

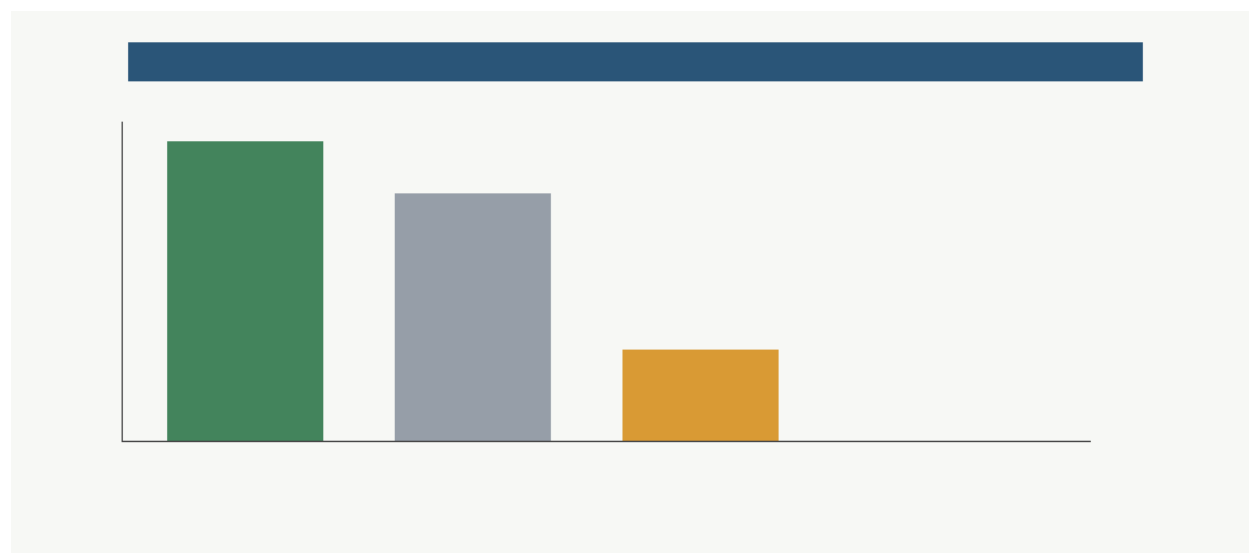


Figure 2: consistency coverage of endpoint invariants across canonical campaign summaries and reports.

The ambiguous text warnings were reader-risk flags, not contradictions. They came from terms such as “winner,” “measured evidence,” or “superiority” appearing in contexts that still negated current physicalized superiority or referred to future measured-evidence requirements. The checker preserved those warnings for review while reporting zero contradictions.

The auditor found one moderate traceability issue: `closure_archive_summary.json` and `toolchain_condition_summary.json` were checked, but the older closure archive manifest did not include those later or self-referential artifacts, so their matrix milestone ownership fell back to `M-INVARIANT-1`. The auditor fixed the checker with explicit milestone ownership for curated core summaries, regenerated outputs, added a regression test, and validated the result. After the fix, endpoint counters remained zero or false and `introduced_new_gate` remained false.

Cycle 31: Public Programmable-Baseline Recency Probe

Cycle 31 created `M-PUBLICBASE-1`, a public baseline recency and materiality screen. The researcher scoped it to official and public sources, with MLCommons material as primary evidence and NVIDIA material as secondary context only. The goal was to determine whether

newer public benchmark evidence should trigger a future programmable-baseline refresh, not whether it reopened a physicalized-weight claim.

The worker added references [11]-[14] and built `physicalized-weights/scripts/public_baseline_recency_probe.py`, tests, a report, source and delta CSVs, a summary JSON, and a figure.

The probe identified MLPerf Inference v6.0 as the latest official release in scope, published by MLCommons on 2026-04-01 [11]. It is newer than MLPerf Inference v5.1, published on 2025-09-09 [12], and newer than the campaign’s earlier MLPerf documentation reference [10]. The official v6.0 result repository was treated as the primary machine-readable source for any future baseline refresh [13]. NVIDIA’s MLPerf page was retained as secondary vendor context only [14].

Source	Role	Machine-readable	Directly usable in current model	Can satisfy measured hybrid reopen
MLPerf Inference documentation [10]	primary context	partial	no	no
MLPerf Inference v6.0 results [11]	primary release page	linked repository	partial	no
MLPerf Inference v5.1 results [12]	primary release page	linked repository	partial	no
MLPerf Inference v6.0 repository [13]	primary result repository	yes	partial	no
NVIDIA MLPerf page [14]	secondary context	no	no	no

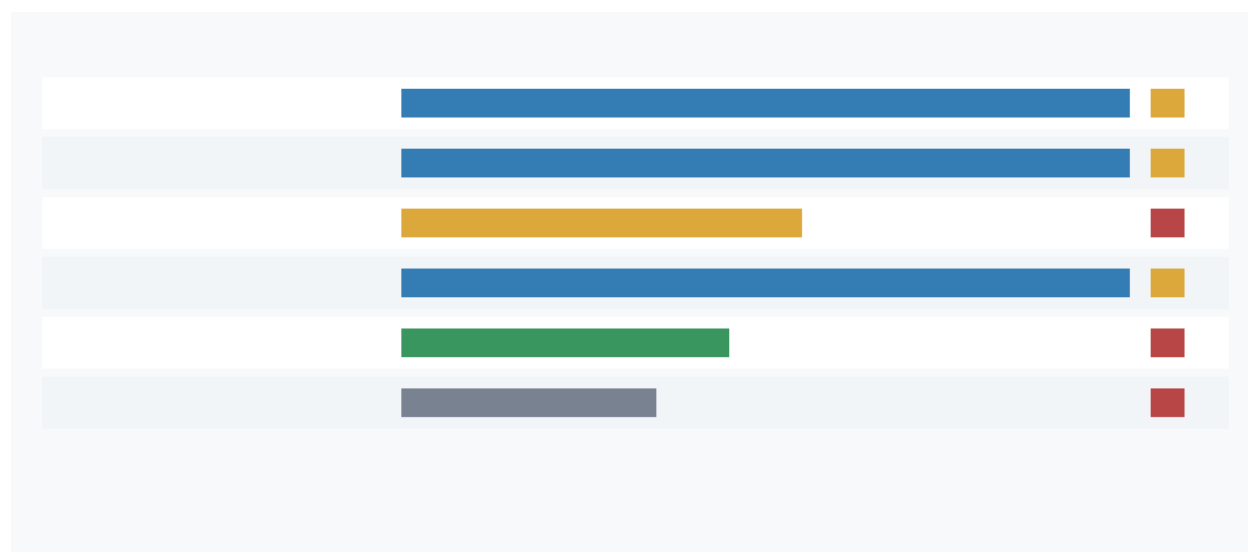


Figure 3: public programmable-baseline recency and materiality screen against campaign calibrated-baseline assumptions.

The final summary values were:

Field	Value
Latest release	MLPerf Inference v6.0
Latest publication date	2026-04-01
Newer than campaign reference	true
Material public baseline update count	3
Model refresh recommended	true
Public sources reopen physicalized claim	false
Current superiority claim count	0
Actual reopen candidate count	0
New reopen gate count	0
Current artifacts reopen	false

The auditor found one moderate source-traceability gap in the first worker build: source rows recorded release identity, dates, URLs, and endpoint effects, but benchmark workloads, hardware-family context, calibration usability, and measured-hybrid-reopen eligibility were mostly in prose. The auditor fixed this by adding structured fields to `public_baseline_sources.csv`, exposing them in the generated report, and adding a regression test. The validated result recommends only a future programmable-baseline prior refresh.

Discussion

Cycles 29-31 changed the handoff and maintenance picture, not the scientific endpoint.

Cycle 29 showed that local prototype verification remains partially refreshed but not fully compiled. Verilator lint, Yosys eval, Yosys synthesis, and Graphviz checks pass. The unrun compiled simulation is an environmental gap caused by missing `make` and C++ compiler support, not a failing HDL result.

Cycle 30 showed that the generated campaign record is internally consistent at the endpoint level. The checker found no machine-readable contradiction across canonical summaries. Its warning-level text scan surfaced phrases that a future reader might review, but none asserted a current physicalized-weight win or actual reopen.

Cycle 31 added the main forward-looking result in this range: public programmable baselines have moved. MLPerf Inference v6.0 is newer than the campaign's prior public MLPerf reference set and is material enough to justify a future programmable-baseline refresh. That direction updates the null hypothesis side of the campaign. It does not supply measured hybrid production, shadow, or canary telemetry under identical workload accounting, so it cannot reopen the physicalized-superiority claim.

Open Questions

Compiled Verilator equivalence remains open until `make` and a C++ compiler are available locally. If those tools appear, rerun the existing toolchain probe and audit compiled HDL vector equivalence. A mismatch would reopen prototype correctness only.

A programmable-baseline refresh is now reasonable but was not performed in these cycles. The validated guidance says that such a refresh should use primary MLCommons repository data and a defensible mapping to the campaign's model terms, including feature extraction, audit, fallback, update cadence, utilization, energy, and latency.

No lifecycle-valid measured hybrid production, shadow, or canary evidence has been ingested. Without that evidence package, the Phase 4 reopen path remains inactive.

References

[10] MLCommons, “MLPerf Inference: Datacenter benchmark documentation,” MLCommons. <https://docs.mlcommons.org/inference/>

[11] MLCommons, “MLPerf Inference v6.0 Results,” MLCommons, 2026. <https://mlcommons.org/2026/04/mlperf-inference-v6-0-results/>

[12] MLCommons, “MLPerf Inference v5.1 Results,” MLCommons, 2025. <https://mlcommons.org/2025/09/mlperf-inference-v5-1-results/>

[13] MLCommons, “MLPerf Inference Results v6.0,” GitHub, 2026. https://github.com/mlcommons/inference-results_v6.0

[14] NVIDIA, “MLPerf AI Benchmarks,” NVIDIA. <https://www.nvidia.com/en-us/data-center/resources/mlperf-benchmarks/>

Appendix: Implementation Details

Code Organization

Cycle 29 added:

File	Lines	Purpose
physicalized-weights/scripts/toolchain_condition_probe.py	382	Local HDL/toolchain capability probe and conditional compiled-Verilator status checker
physicalized-weights/tests/test_toolchain_condition_probe.py	125	Tool availability, blocked compiled-Verilator status, refreshed HDL checks, endpoint-counter, and PNG tests
physicalized-weights/docs/toolchain_condition_report.md	58	Tool availability, refreshed HDL check, compiled-Verilator blocker, and prototype-only interpretation report

Cycle 30 added:

File	Lines	Purpose
physicalized-weights/scripts/campaign_invariant_checker.py	427	Cross-artifact endpoint-invariant consistency checker

File	Lines	Purpose
physicalized-weights/tests/test_campaign_invariant_checker.py	141	Zero-contradiction, endpoint-counter, synthetic contradiction, warning, no-new-gate, and ownership-regression tests
physicalized-weights/docs/campaign_invariant_report.md	54	Invariant coverage, contradiction count, ambiguity warnings, and no-new-gate report

Cycle 31 added:

File	Lines	Purpose
physicalized-weights/scripts/public_baseline_recency_probe.py	399	Public programmable-baseline recency and materiality screen
physicalized-weights/tests/test_public_baseline_recency_probe.py	143	MLPerf recency, source traceability, model-refresh, non-reopen, endpoint-counter, and PNG tests
physicalized-weights/docs/public_baseline_recency_report.md	48	Official MLPerf recency, source table, baseline materiality, and future-refresh boundary report

Generated cycle 29-31 data and figures:

Artifact family	Files
Toolchain condition	toolchain_condition_matrix.csv, toolchain_condition_summary.json, toolchain_condition_matrix.png, toolchain_verilator_lint.log, toolchain_yosys_eval.log, toolchain_yosys_synthesis.log, toolchain_graphviz.log
Campaign invariant checker	campaign_invariant_matrix.csv, campaign_invariant_summary.json, campaign_invariant_matrix.png
Public baseline recency	public_baseline_sources.csv, public_baseline_delta_matrix.csv, public_baseline_recency_summary.json, public_baseline_delta_matrix.png

All three generated figures were valid PNG files, each 960 x 420, 8-bit RGB.

Test Results

Validation commands reported by the worker and auditor sessions:

```
python3 physicalized-weights/scripts/toolchain_condition_probe.py
python3 physicalized-weights/tests/test_toolchain_condition_probe.py
file physicalized-weights/data/toolchain_condition_matrix.png

python3 physicalized-weights/scripts/campaign_invariant_checker.py
python3 physicalized-weights/tests/test_campaign_invariant_checker.py
file physicalized-weights/data/campaign_invariant_matrix.png

python3 physicalized-weights/scripts/public_baseline_recency_probe.py
python3 physicalized-weights/tests/test_public_baseline_recency_probe.py
file physicalized-weights/data/public_baseline_delta_matrix.png

python3 -m long_exposure.tools.promise_check .
python3 -m long_exposure.tools.org_check .
```

Final validator state after cycle 31:

Check	Result
promise_check	exit 0, events: 83, plan milestones: 31
org_check	exit 0
Remaining promise_check warnings	known orphan report artifacts in reports/cycles/
Remaining org_check warnings	known root prompt and live log files

Manifest Snapshot

MANIFEST.md was updated for this report cycle. The current snapshot records:

Count	Value
Total authored research scripts	33
Total authored research script lines	14,385
Total authored tests	30
Total authored test lines	4,164
Total authored HDL/support files	4
Total authored HDL/support lines	241
Total authored research docs and diagram sources	33
Total authored doc/source lines	2,061
Toolchain condition matrix rows	10
Campaign invariant matrix rows	49
Campaign invariant contradictions	0
Public baseline source rows	5
Public baseline delta rows	6
Ledger events	83
Plan milestones	31

No ## Key Files section was present in the manifest, so no final-reporter-owned section required preservation.

Session References

Role	Session ID	Contents
Cycle 29 researcher	6e9bf19a-674b-4e30-a515-d856be1f72a3	Scoped M-T00LCHAIN-1 as a prototype-verification refresh, not a performance reopen
Cycle 29 worker	8f6dfd72-8a58-4446-b03a-87d07034faf2	Built and ran the toolchain condition probe
Cycle 29 auditor	03c7cbc6-ad5f-4254-873f-b7c85e6527be	Validated M-T00LCHAIN-1 with no critical or moderate issues
Cycle 30 researcher	e48b5978-e6df-44b5-b87b-c1bfe4b89c1b	Scoped M-INVARIANT-1 as cross-artifact endpoint QA
Cycle 30 worker	a48d9de1-8a60-4e9c-b973-4320dd867939	Built and ran the campaign invariant checker
Cycle 30 auditor	8347638a-ea61-4f8f-9c84-1df555def8ac	Validated after fixing milestone-ownership traceability
Cycle 31 researcher	6b47f02d-c4f8-4f3a-b687-79b01bfddae4	Scoped M-PUBLICBASE-1 as a public programmable-baseline recency screen
Cycle 31 worker	52232565-36e9-4f38-bdb8-8af0b52e503e	Built and ran the public baseline recency probe and updated references
Cycle 31 auditor	c445fb5f-63d1-45fc-9a62-809e65368003	Validated after fixing structured source traceability

Cross-Reference Map

verify_prototype_closure.py, safety_filter_core.sv, run_yosys_eval.py, safety_filter_core.py, and safety_filter_core_tb.cpp feed toolchain_condition_probe.py. That probe refreshed existing prototype evidence and documented the compiled-simulation blocker.

Canonical Phase 2, Phase 3, Phase 4, robustness, deferral, closure, archive, and toolchain summaries feed campaign_invariant_checker.py. That checker confirmed endpoint consistency without adding a new scientific criterion.

REFERENCES.md, stronger_baseline_summary.json, stronger_baseline_comparison.csv, and phase2_synthesis_summary.json feed public_baseline_recency_probe.py. That probe screened newer official MLCommons material for programmable-baseline drift while preserving no measured hybrid reopen.